PERSONAL PROJECT · SCIENTIFIC COMPUTING

LLM Resource Allocation

A verified reference and CPU performance study

Antonio Arbizzani Tonelli

linkedin.com/in/antonio-arbizzani-tonelli

Python/Pyomo reference, serial C++20 port and OpenMP campaigns for a published allocation model. Formulation and GH/AGH algorithms: Cheng and Nguyen, arXiv:2604.07472.

github.com/antonio-arbizzani-tonelli/llm-resource-allocation · MIT license · October 2026

OVERVIEW · RESULTS AT A GLANCE

Four measured results, each with its scope

63/63

Cross-language checks pass: 3,023 final values bitwise identical

48 placement cases (4 frozen) and 15 seeded-shuffle checks · bounded synthetic verification.

120.54×

Python/C++ core-timer ratio of medians, AGH 16×4×4, load 200

95% CI [109.75, 131.22] · complete invocation 99.70× · not a pure language effect.

4.02×

OpenMP throughput, 8 threads vs one thread, fixed 32-case batch

95% CI [3.76, 4.42] · 50.28% efficiency · 30 paired rounds · each solve stays serial.

1.94

Log-log slope of native AGH time: about 3.8× per doubling of queries

95% CI [1.93, 1.96] · 16–384 queries, five input seeds each · GH 1.66 · descriptive, not asymptotic.

Synthetic inputs, CPU only (i7-13620H, WSL) · GPU latencies are model coefficients · sources in Appendix A8.

01 · THE PROBLEM

Allocation under service constraints

Choose model deployments and route demand across heterogeneous GPU options while meeting budget, capacity and service-level constraints.

Workload

Demand per query type, forecast or realized.

Allocator

Chooses deployments, GPU counts, parallelism and routing.

Budget · Capacity · SLOs

Models and GPU options

Heterogeneous options, modeled rather than measured on a cluster.

THIS PROJECT

Implements and verifies the allocation software, then measures its CPU runtime.

Formulation and GH/AGH: Cheng and Nguyen, arXiv:2604.07472 v2 (v1 for the operational LP).

02 · MODEL

The allocation model: two decision stages

STAGE 1 · PLANNING

Choose deployment and nominal routing

VARIABLE	GPU counts, model placement, TP/PP, routed and unmet fractions.
FIXED	Forecast demand and model coefficients.

STAGE 2 · OPERATION

Adapt routing to a supplied scenario

VARIABLE	Routed and unmet fractions, via the operational LP.
FIXED	GPU counts, admissions and TP/PP from Stage 1.

Objective · GPU rental C1 + model storage C2 + data storage C3 + delay C4 + unmet demand C5

Constraints · demand balance, admission, compute, memory, storage, latency, error, budget on C1+C2+C3

GPU capacity and latency are model inputs · all runtime experiments execute allocation software on CPU.

03 · PYTHON REFERENCE

Python and Pyomo as the scientific reference

Inputs
Validated problem data with explicit units.

→

Algorithm or model
GH/AGH in Python, or a Pyomo MILP/LP solved by HiGHS or Gurobi.

→

Decision record
Plain domain data; extraction never repairs a decision.

→

Independent validator
Recomputes constraints and objective; accepts or records failure.

KEPT AS SEPARATE EVIDENCE

- **Completion**
- **Constraint feasibility**
- **Objective agreement**
- **Solver termination**

SOLVER BOUNDARY

Pyomo builds the model; the configured backend solves it. A time-limited incumbent is not a proven optimum, and a heuristic has no optimality certificate.

Roles from docs/ARCHITECTURE.md and src/ · implementation responsibilities, not performance claims.

04 · C++20 PORT

C++20 implements the same Stage-1 heuristics

Component	Python reference	C++20 native
GH and AGH (Stage 1)	Reference implementation	Serial port, same decisions
Costs, traces, validation	Independent validator	Native checks; output re-validated in Python
P_DM exact MILP	Pyomo + HiGHS or Gurobi	Future work
Stage-2 operational LP	Pyomo + HiGHS or Gurobi	Future work
Experiment scheduling	Python experiment runners	OpenMP across independent cases

SAME NUMERICAL RULES

Explicit owned state, deterministic ordering, Python's summation order; no fast-math or FP contraction.

SEPARATE RESPONSIBILITIES

Input preparation, algorithms, validation and campaign scheduling live in separate modules.

Source: cpp/README.md and docs/ARCHITECTURE.md · commit 80a9625.

05 · VERIFICATION

Verification before performance comparison

63/63

cross-language checks pass: 48 placement cases (4 frozen) and 15 seeded-shuffle checks.

3,023

compared final numeric values, with zero bitwise differences.

TWO DISTINCT CHECKS

Python and C++ decisions compared value by value, plus independent recomputation of constraints and objective.

STATED COVERAGE

Historical AGH Python runs at 256×4×4 and 128×8×8 failed with MemoryError: final-decision checks only, no Python time. In the scaling study, full parity covers 20 small points; 70 others have independent final-decision validation.

WHAT PARITY SHOWS

The port matches the reference. It does not prove the reference matches the paper.

Source: release parity check · bounded synthetic verification · commit 80a9625.

06 · MATCHED RUNTIME

Matched runtime: core and complete invocation

AGH, 16 queries × 4 models × 4 GPU tiers, load 200 · same WSL environment · ten paired rounds per timer.

Core timer · medians: Python 8.425 s, C++ 0.0699 s



120.54×

CI [109.75, 131.22]

Complete invocation · medians: Python 9.884 s, C++ 0.0991 s



99.70×

CI [93.26, 107.26]

Bars: Python/C++ ratio of medians, common scale. Whiskers: 95% paired-round percentile bootstrap, 10,000 resamples.

TWO SEPARATE TIMERS

Core: fresh public solve with traces and validation; excludes startup, parsing and output. **Invocation:** a Linux subprocess from startup to exit.

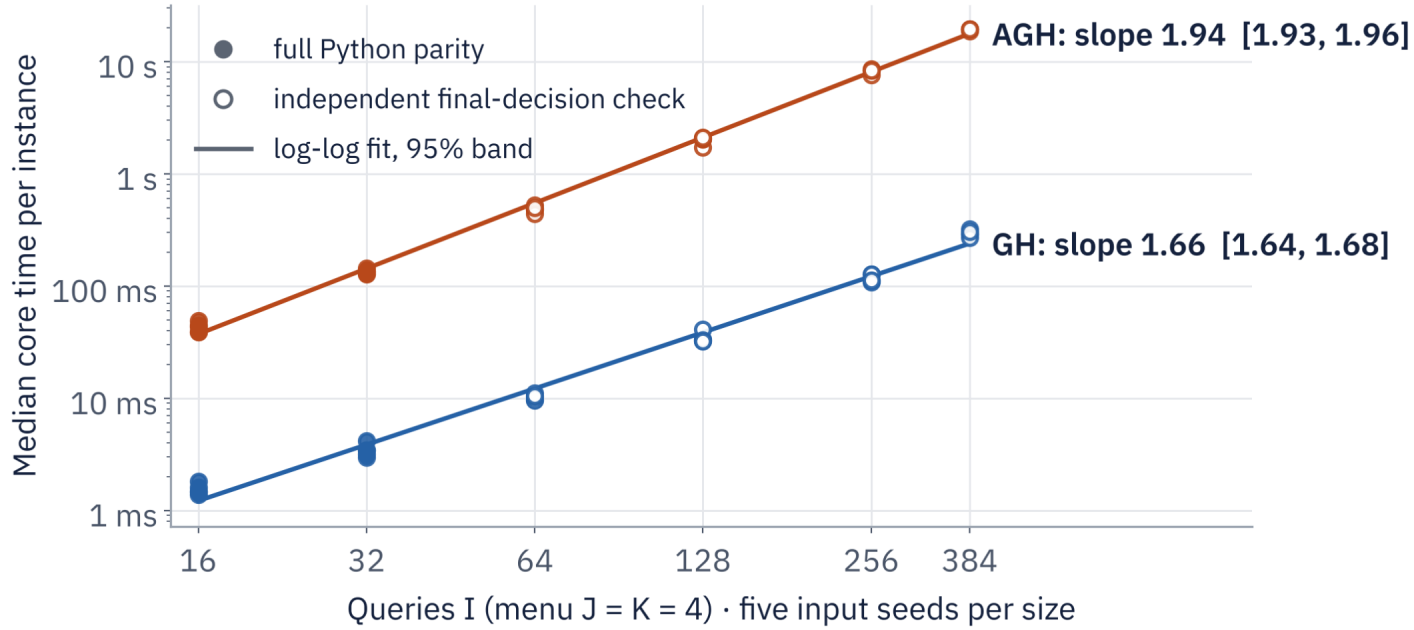
NOT A PURE LANGUAGE EFFECT

Python validates input and builds PdmProblemData inside its timed call; C++ receives a prepared coefficient lattice. Trace costs also differ.

Source: WP1 matched-runtime analysis · i7-13620H, WSL · commit 80a9625.

07 · SCALING

Native scaling across seeded instances



Filled points: full Python parity (16 and 32 queries). Open points: independent final-decision checks. Fits describe the measured range, not asymptotic complexity.

GH slope

1.6615

CI [1.6402, 1.6798] · R^2 0.9903 · ≈3.2× time per doubling

AGH slope

1.9402

CI [1.9254, 1.9600] · R^2 0.9975 · ≈3.8× time per doubling

DESIGN

Six sizes, five input seeds each · median of three core solves per instance.

Source: WP5 instance-scaling analysis · 5 seed clusters, 10,000 draws · commit 80a9625.

09 · OPENMP DIAGNOSTICS

Diagnosing the lost efficiency

6,030/6,030
observations: 201 settings × 30 rounds

6.23%
wall-idle proxy at 8 threads

14.13%
Amdahl-equivalent serial fraction (a fit)

4.55×
16 threads, CI [3.94, 5.02], 28.46% efficiency

Hypothesis	Intervention or measurement	Outcome
Load balance	Schedule, chunk and case-ordering interventions; per-worker wall time	Sorting gain unestablished: 1.0053 [0.959, 1.200] at 16 threads
Memory allocator	MALLOC_ARENA_MAX=1 at 8 threads, same setting	Much slower: median 2.989 s vs 0.251 s
Thread placement	Places and binding levels at dynamic, chunk 1	One exploratory gain: cores/spread at 4 threads, 1.138 [1.070, 1.264]
Hardware causes	P/E cores, SMT, cache bandwidth, frequency	Unresolved: WSL hides topology; no counters or frequency series

Source: WP2 OpenMP analysis · unadjusted exploratory intervals · commit 80a9625.

10 · SKILLS

What this project demonstrates

Optimization modeling

Pyomo MILP and LP models solved with HiGHS or Gurobi; GH and AGH heuristics; explicit units.

C++20 engineering

Serial GH/AGH port with deterministic ordering and no fast-math; release and ASan/UBSan checks.

Verification

Independent validator; 63/63 cross-language checks with 3,023 values bitwise identical.

Performance measurement

Separate timers, paired rounds and bootstrap intervals; OpenMP across independent cases.

Reproducibility

Seeded inputs, replay commands and a SHA-256 manifest of 3,572 published files.

Honest reporting

Negative and open results kept: allocator slowdown, unestablished sorting gain, rejected incumbents.

What is done, what is measured, what is open

IMPLEMENTED

- Python/Pyomo reference and independent validator
- Serial C++20 GH and AGH
- OpenMP across independent cases
- Opt-in relative Stage-2 objective tolerance

MEASURED

- Local CPU runtime on synthetic inputs
- Separate core and invocation timers
- Paired rounds and bootstrap intervals
- Stated verification coverage

STILL OPEN

- Author-data calibration
- Native exact and Stage-2 solvers
- Memory profiling
- A validated MPI implementation
- Physical causes of OpenMP efficiency loss

NOT CLAIMED

GPU-serving speed, real cloud savings, heuristic optimality or replication of the paper's tables.

REPOSITORY · MIT LICENSE · CONTACT

github.com/antonio-arbizzani-tonelli/llm-resource-allocation

linkedin.com/in/antonio-arbizzani-tonelli

APPENDIX A1 · UNITS

An explicit dimensional convention

ONE CONVERSION, SPELLED OUT

KV GB per GPU = β · r · T_res · (λ / 3600) · v / (TP · PP)

λ is in requests per hour and T_res in seconds, so the factor 3600 turns arrivals into average concurrent requests.

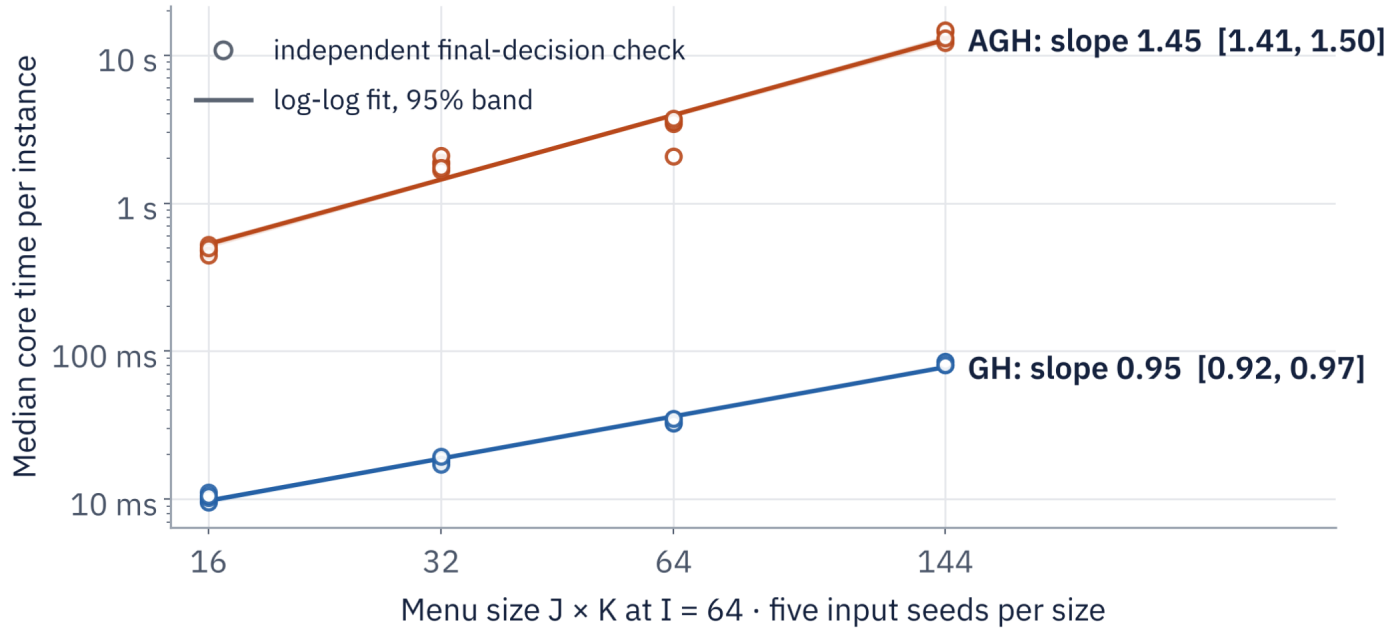
The historical printed arithmetic stays available and clearly labeled as LITERAL_PAPER; new instances are COHERENT.

Quantity	Unit
Arrival rate λ	requests/hour
KV residency	seconds/request
Data residency	hours
Horizon H	hours
Processing delay	seconds/request
Delay penalty	USD/(ms·request)

The dimensional reading is a project choice, not a claim about what the paper printed.

APPENDIX A2 · MENU SCALING

Larger model and GPU-tier menus at 64 queries



Menus 4×4, 8×4, 8×8 and 12×12 at I = 64 (products 16–144). Every point has an independent final-decision check.

GH slope

0.9455

CI [0.9185, 0.9707] · R^2 0.9928

AGH slope

1.4461

CI [1.4065, 1.4998] · R^2 0.9694

LIMIT

Menu aspect ratios and structural branches confound a single product exponent.

Source: WP5 instance-scaling analysis · descriptive observed-range fits · commit 80a9625.

APPENDIX A3 · SOLVER BUDGETS

More solver time changes the ordering on one input

10×10×10, LOAD 1 · ACCEPTED MODELED COST, USD



18/20
measured jobs accepted (20/20 completed) on four fixed inputs, fresh cold solves

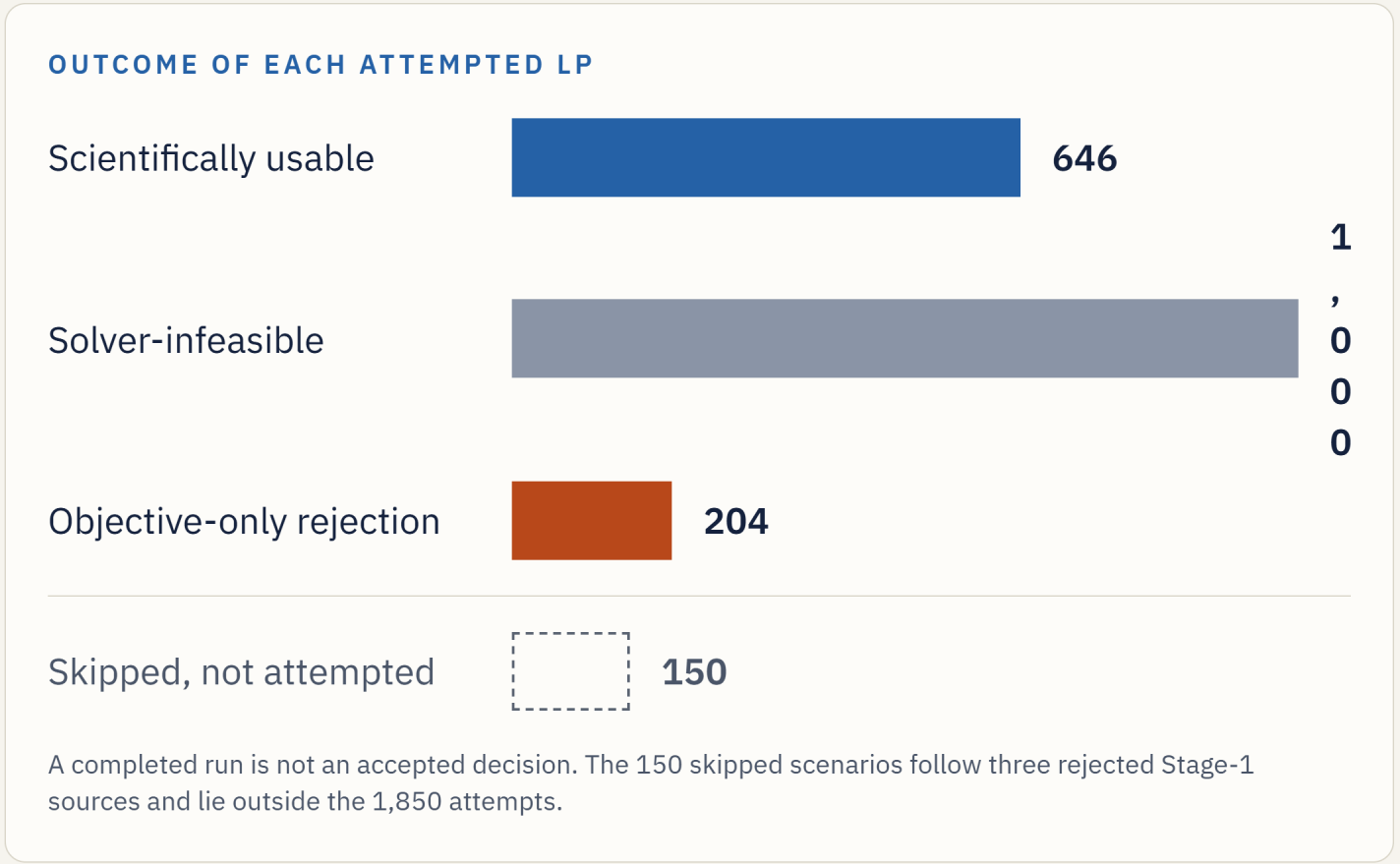
12/12
MILPs end at the time limit; normalized gaps 0.987–0.99999998: weak bounds, no optimum

0.845 → 2.103
median AGH/HiGHS cost: 20 s (4/4 pairs) vs 120 and 600 s (3/4 pairs); cohorts differ

Source: WP6 solver-quality analysis · HiGHS only: Gurobi WSL license unavailable · commit 80a9625.

APPENDIX A4 · STAGE 2, ORIGINAL CAMPAIGN (OCT 13)

Standard policy: 1,850 operational attempts



A completed run is not an accepted decision. The 150 skipped scenarios follow three rejected Stage-1 sources and lie outside the 1,850 attempts.

37/40
Stage-1 sources accepted; all 40 jobs completed

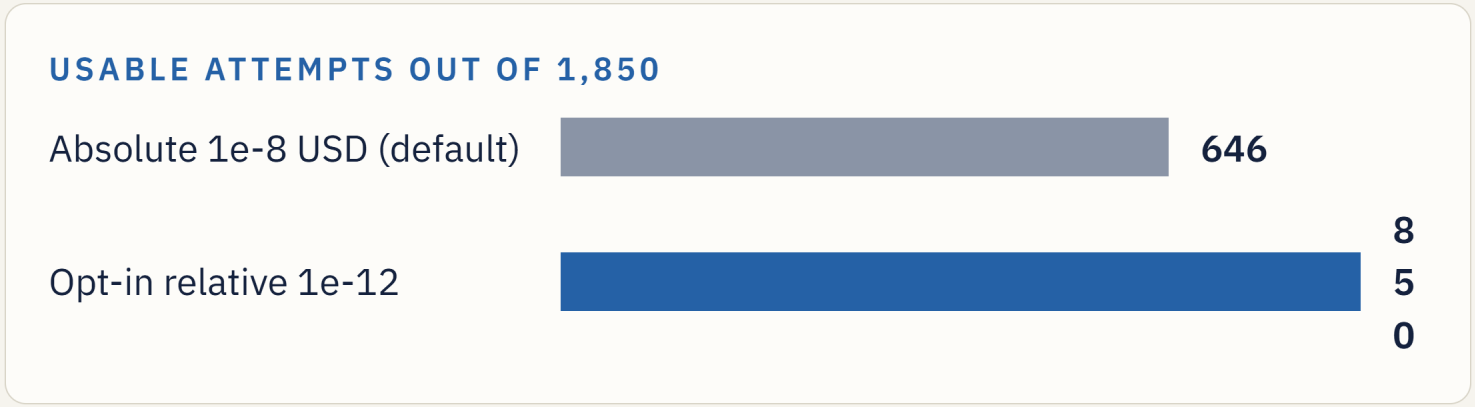
19.05%
256 of 1,344 frozen-routing failures recovered by adaptation

5.86%
summed C4+C5 reduction over 390 usable pairs; no savings for infeasible baselines

Source: original Stage-2 analysis · absolute 1e-8 USD objective agreement · commit 80a9625.

APPENDIX A5 · RETROSPECTIVE POLICY REPLAY

Same decisions, optional relative tolerance



Policy	Recovered	C4+C5 reduction, 95% CI (usable pairs)
Absolute default	256/1,344	5.86% [5.18, 6.12] (390)
Opt-in relative	344/1,344	5.96% [5.75, 6.06] (506)

Rule |a−b| ≤ max(abs_tol, rel_tol·max(|a|,|b|)). No LP solved, no decision repaired; defaults unchanged. Different cohorts: not a causal policy benefit.

204/204
objective-only rejections accepted: 1–3 floating-point spacings apart

1,000
solver-infeasible attempts still rejected

~2e-6
injected relative objective error is still rejected

Source: WP3 policy-replay analysis · retrospective, no new solves · commit 80a9625.

APPENDIX A6 · STAGE 2, NEW CAMPAIGN (OCT 14)

New matched solves across four loads

3,246/6,800
usable LPs; 3,554 solver-infeasible

136/144
sources accepted, all 144 complete

400
planned LPs unattempted: 8 rejected P_DM sources

40/48
inputs accepted for all methods: no ranking

Nominal load	Recovered / frozen failures	Usable cost pairs	C4+C5 reduction, 95% CI
200	330/1,260 (26.19%)	540	4.95% [4.64, 5.18]
2,000	280/1,260 (22.22%)	540	4.92% [4.63, 5.15]
20,000	400/1,220 (32.79%)	530	5.16% [4.73, 5.53]
100,000	311/1,135 (27.40%)	315	5.43% [4.98, 5.87]

Explicit relative objective tolerance 1e-12, HiGHS, 20 s limits. Reduction = (Σ frozen − Σ adapted) / Σ frozen on identical usable pairs; infeasible baselines get no savings, and lower modeled cost may come with more unmet demand.

Source: WP4 Stage-2 analysis · 12 input clusters per load · conditional synthetic grid · commit 80a9625.

APPENDIX A7 · EVIDENCE SETS

Earlier results keep their own scope

Evidence set	Headline	Why it stays separate
Release parity suite	63/63 checks · 3,023 values bitwise	Bounded fixture suite: 48 placement, 15 shuffle
Full comparisons, Oct 13	16 comparisons · 185,832 values bitwise	Two AGH MemoryError points: final-decision only
Cross-platform timing, Oct 13	103.93× AGH 16×4×4, load 200	Windows Python vs WSL C++, different timers
OpenMP campaign, Oct 13	4.20× core · 2.27× invocation	18 rounds; not combined with the 30-round study
OpenMP 16-case, Oct 11–12	5.16× core, 16 large AGH	Minima of three rounds, different batch
Allocation quality, Oct 11–12	AGH vs GH rental C1: 9 → 8 USD	C1 −11.1%; total score only 0.087–0.278%
Validator probe, Oct 11–12	Parallel validator slower than serial	Negative result kept; validation stays serial
Checkpoint 543ad4e	1,131 Python tests; ASan/UBSan	Development tree; public tree omits one module

Sources: docs/RESULTS.md, PERFORMANCE.md, VALIDATION.md, KEY_NUMBERS.md · commit 80a9625.

APPENDIX A8 · SOURCES

Claim-to-source table

Paths under benchmarks/ at public commit 80a9625ab62a3c5c70ede7a9dae97a61158e250a; its numerical payload matches checkpoint 543ad4e.

Study (slides)	Artifact	Selector
Release parity (7)	portfolio/raw/serial/parity.json	passed, total; Σ compared_numeric_values
WP1 runtime (8)	matched_runtime_2026-10-14/analysis/analysis.json	case 16x4x4_load200_agh; core, invocation
WP5 scaling (9, A2)	instance_scaling_2026-10-14/analysis/analysis.json	fits: family query/menu, method gh/agh
WP2 OpenMP (10, 11)	openmp_diagnostics_2026-10-14/analysis/extension.json	key 8_cores_close_dynamic_1_False_None
WP6 solver (A3)	solver_quality_2026-10-14/analysis.json	rows 10x10x10_L1_agh, _highs_20/120/600
Stage 2, Oct 13 (A4)	stage2_stress_2026-10-13/analysis/analysis.json	overall: status counts, recovery, pairs
WP3 replay (A5)	stage2_policy_2026-10-14/analysis.json	old_policy, selected_policy
WP4 Stage 2 (A6)	stage2_robustness_2026-10-14/analysis.json	sources, status_counts, by_load

Replay commands: docs/KEY_NUMBERS.md · interpretation: EXTENSION_REPORT.md, MODEL.md, ARCHITECTURE.md, VALIDATION.md.